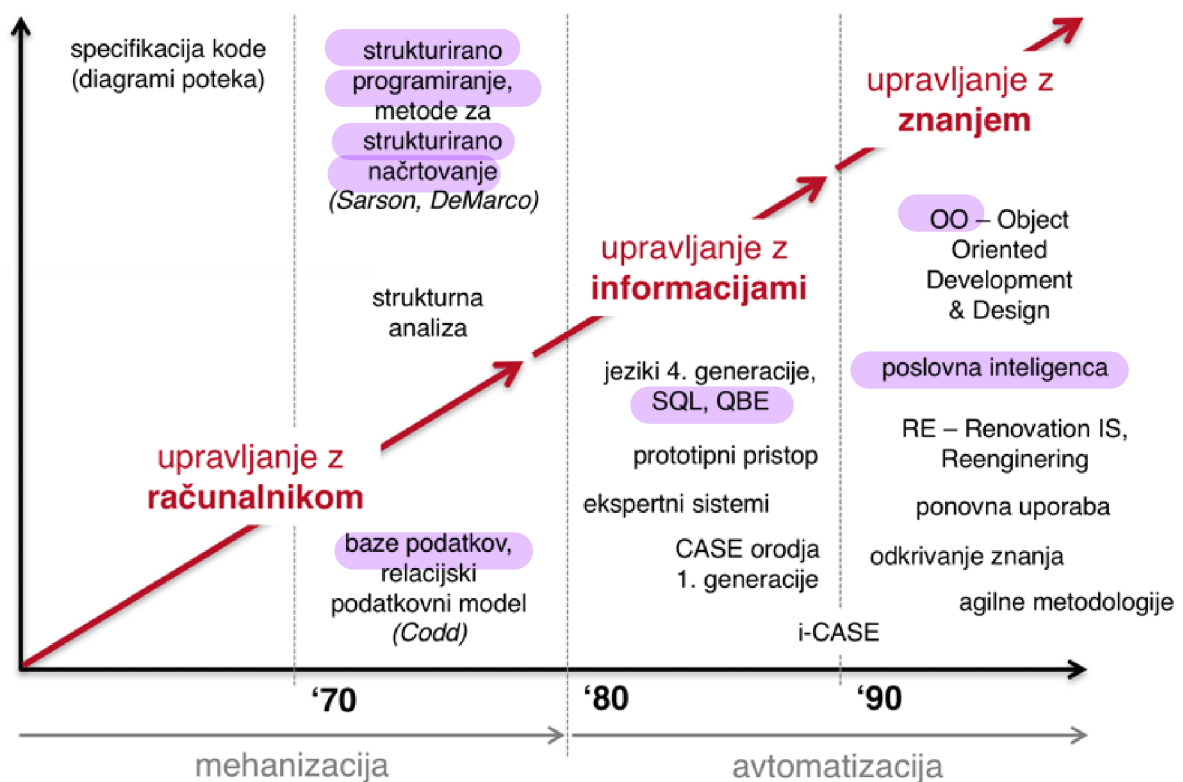


Poglavje 14 **P7** Razvojni modeli informacijskih sistemov in metodologije razvoja



P7

14.1 Zgodovina razvoja informacijskih sistemov



Slika 14.1: Zgodovina razvoja informacijskih sistemov

Vsakdo lahko žanje **sadove naključnega uspeha**, vendar naj jih **ne uporabi za temelje svojega dela v prihodnje**. (Bicknel, 1993)

14.2 Razvojni model informacijskega sistema

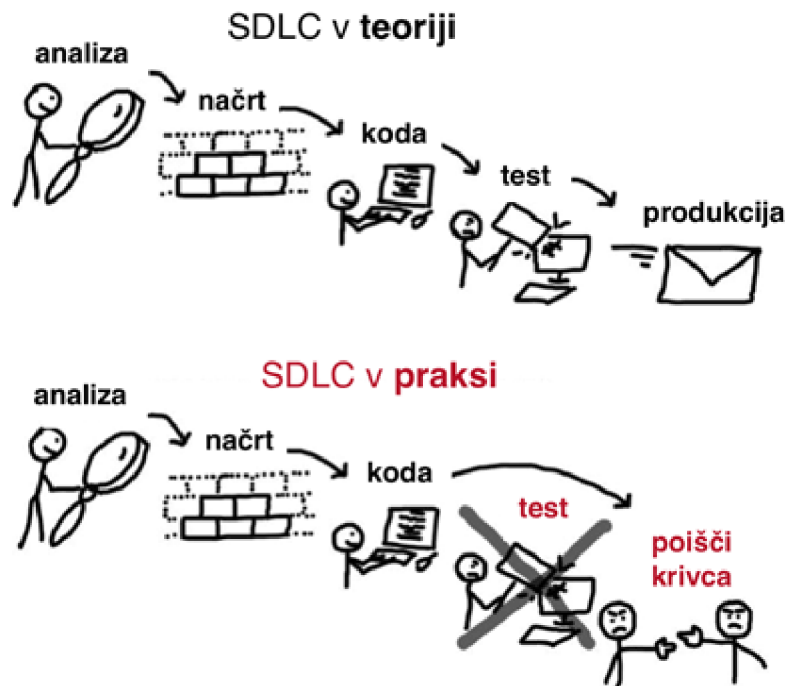
Razvojni model informacijskega sistema (angl. *System Development Life Cycle (SDLC)*) pove, v kakšnem **sosledju** in na kakšen **način** si v okviru razvoja informacijskega sistema **sledijo posamezne faze**.

System Development Life Cycle (SDLC)-Natasha Evelyn Dickson Yongsen



Razvoj informacijskega sistema zajema številna opravila, po navadi razdeljena v **faze** (Avison and Fitzgerald 2006):

- **študija izvedljivosti** (glej poglavje 14.2.1),
- **zbiranje zahtev** (glej poglavje 14.2.2),
- **analiza** (glej poglavje 14.2.3),
- **načrtovanje** (glej poglavje 14.2.4),
- **implementacija** (glej poglavje 14.2.5),
- **pregled in vzdrževanje** (glej poglavje 14.2.6). evolucija



Slika 14.2: SDLC v teoriji in praksi

14.2.1 Študija izvedljivosti

Prva aktivnost je **študija izvedljivosti** (angl. *feasibility study*), kjer pregledamo obstoječe stanje:

- **obstoječe zahteve**, ki naj bi bile zadoščene,
- **težave pri izpolnjevanju zahtev**, ponavadi deluje prepočasi
- **nove zahteve**, treba je updatat
- kratek **pregled alternativ**, preverimo ideje in razložimo zakaj bi to delalo, kaj je + in kaj -

Vsak predlagan sistem mora biti **izvedljiv z naslednjih vidikov**:

- **pravnega** (ni v konfliktu z nobenim zakonom), - pri globalnih imokementacijah pride do velikih težav, verjetno ne bo upošteval zakonov vseh držav
- **organizacijskega** (je sprejemljiv za podjetje, še posebej, če gre za uvajanje večjih sprememb),
- **tehničnega** (lahko ga podpremo z obstoječo tehnologijo in imamo na voljo tudi tehnično znanje za implementacijo) in a je na voljo dovolj tehnologije, ali imamo dovolj znanja in kadra
- **ekonomskega** (finančno sprejemljiv in stroški so upravičeni).

14.2.2 Zbiranje zahtev

smo ga predszavili, so ga odobrili itd.

Ko **vodstvo odobri** projekt, začnemo **z zbiranjem zahtev** (angl. *systems investigation*) z namenom pridobitve podrobnejših informacij o:

- **funkcionalnih zahtevah** (angl. *functional requirements*), funkcionalnosti, ki naj bi jih imel sistem, ki naj bi jih omogočal nefunkcionalnosti - kaj je lahko narobe, kje so lahko težave (100 hkratnih uporabnikov)
- zahtevah novega sistema,
- vpeljanih novih omejitev,
- izjemah in
- težavah obstoječih postopkov dela.

Dejstva se pridobijo s **strani zaposlenih** (vodstveni in operativni delavci) s pomočjo **različnih pristopov**:

- **opazovanje** (vpogled v težave, delovne pogoje, ozka grla),
- **intervjuji** (individualni ali skupinski, ena najbolj zanesljivih metod),
- **vprašalniki** (po navadi, ko želimo pridobiti odgovor množice uporabnikov ali iz oddaljenih lokacij),
- **pregled dokumentacije** (lahko identificiramo težave, vendar se moramo zavedati, da je **dokumentacija** pogosto pomanjkljiva) idr.

14.2.3 Analiza

Ko imamo zbranih dovolj informacij, začnemo z **analizo** (*angl. **W** systems analysis*), pri kateri si postavljamo naslednja vprašanja:

- Zakaj ti problemi obstajajo?
- Zakaj uporabljamo določene metode dela?
- Ali obstajajo alternativni načini dela?
- Kakšna je verjetnost povečanja rasti količine podatkov?
- idr.

Razumeti poskušamo, **zakaj je obstoječi sistem zgrajen tako kot je in identificirati možne izboljšave** v okviru predlaganega sistema.

14.2.4 Načrtovanje

Na podlagi rezultatov prejšnjih korakov nadaljujemo z **načrtovanjem** (*angl. **W** systems design*), kjer pripravimo načrt računalniških in ostalih komponent sistema:

- **vhodni podatki** in na kakšen način jih **zajamemo**, only-one-principle (stvari zajamemo le enkrat)
 - **izhodi iz sistema**,
 - **procesi** (večina računalniško podprtih), ki skrbijo za **pretvorbo vhodov v izhode**,
 - **struktura podatkov**, ki jih sistem uporablja,
 - **politika dostopa in izdelava varnostnih kopij**, kaj je dostopno, kdo je admin...
 - **testiranje sistema**, osnova
 - **načrt implementacije**.
- } kako se podatki shranijo

14.2.5 Implementacija

Pri **implementaciji** (*angl. **W** implementation*) so pomembni številni elementi:

- **poskrbeti za vse vidike pred vpeljavo sistema** (nakup strojne opreme, razvoj lastne programske opreme, nakup morebitnih knjižnic idr.),
- **vzdrževati kakovost** (testiranje s strani različnih akterjev – končni uporabniki, analitiki itd.), test integracije - kako se vzdržuje sistem
- **izobraževanje uporabnikov** (končnih in podpornih),
- **dokumentacija** (uporabniška in tehnična navodila idr.),
kako uporabljati sistem kako vzdrževati sistem, kako backupat...

- **varnostni testi** (nepooblaščen dostop, vzpostavitev stanja po napaki, obdobje vzporednega delovanja),
- **testi sprejemljivosti** (sistem preide v polno rabo).

14.2.6 Pregled in vzdrževanje

Zadnji korak je **pregled in vzdrževanje** (*angl. [W review and maintenance](#)*) in nastopi, ko je sistem že v uporabi:

- **potrebne spremembe** zaradi boljše učinkovitosti, napredkov tehnologije, dodatnih funkcionalnosti ipd.,
- **pregled funkcionalnosti in skladnosti z zahtevami**,
- **primerjava** ocenjenih in dejanskih **stroškov**,
- idr.

14.3 Metodologija kako faze izvedemo

Obstajajo številne različice razvojnega modela informacijskega sistema in vsaka **metodologija** (*angl. [W methodology](#)*), ki opredeljuje razvojni model, mora vsebovati:

- **faze** (npr. od študije izvedljivosti do pregleda in vzdrževanja, za katere se pričakuje, da jih izvedemo v zaporedju in da ima vsaka faza določene izdelke (npr. dokumenti, izvorna koda itd.)),
- **tehnike** (načini za ocenjevanje stroškov, za podrobno opredelitev načrta itd.),
- **orodja** (programski paketi, ki podpirajo določen del postopka),
- **postopek izobraževanja** (način **uvajanja** novincev).

14.3.1 Tehnike

Poznamo različne tehnike:

- **diagram poteka** (*angl. [W flowchart](#)*),
- **organizacijski diagram** (*angl. [W organizational chart](#)*),
- **mrežni diagram**,
- **specifikacija** zapisov itd.

Najpogosteje uporabljeni tehniki sta:

- **diagram toka podatkov** (*angl. [W Data Flow Diagram \(DFD\)](#)*), ki je pomemben pri **procesnem pogledu** na problem in
- **entitetno-relacijski diagram** (*angl. [W Entity-Relationship Diagram \(ERD\)](#)*), ki je pomemben pri **podatkovno usmerjenem pogledu** na problem.

14.3.1.1 Kako narišemo DFD?

How to Draw Data Flow Diagram?



14.3.1.2 Kako narišemo ERD?

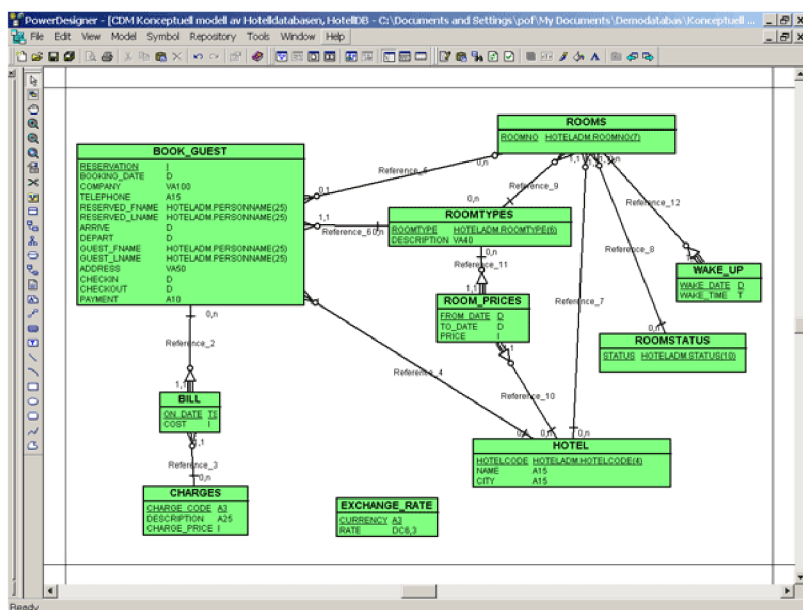
ER- Diagram Explanation Google student club activity 3 subject DBMS CSE Btech



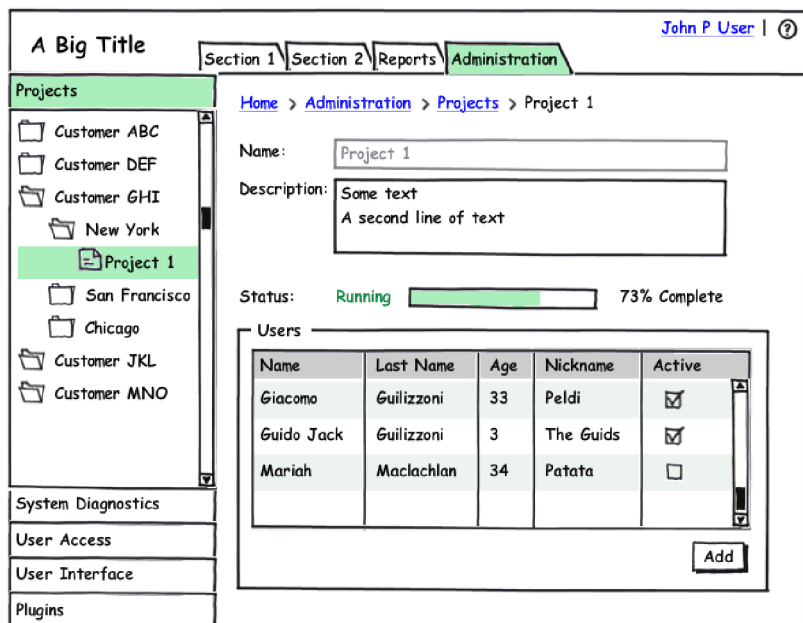
14.3.2 Orodja

Podpora posameznemu oz. več postopkom, kjer je zaželeno, da imajo takšna orodja podporo repozitorijem (za ponovno uporabo gradnikov):

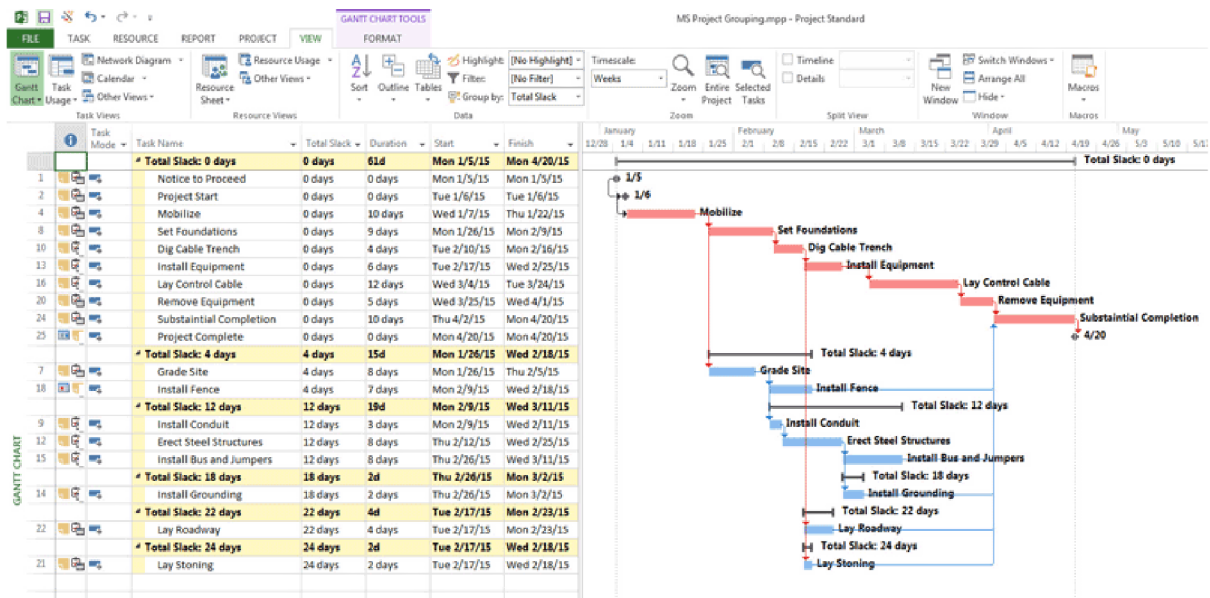
-  PowerDesigner (glej sliko 14.3),
-  Balsamiq Wireframes (glej sliko 14.4), risanje poti na zaslon
-  Pencil Project,
-  Microsoft Visio, risarsko orodje, ki omogoča del funkcionalnosti, ki ga omogoča powerdesign
-  PlantUML, spletna storitev, kjer so podprte vse tehnike diagramov...
-  Microsoft Project (glej sliko 14.5), za projektnoodenje
- idr.



Slika 14.3: Primer načrtovanja podatkovnega modela z orodjem PowerDesigner



Slika 14.4: Primer načrtovanja uporabniškega vmesnika z orodjem Balsamiq Wireframes



Slika 14.5: Primer projektnega vodenja z orodjem Microsoft Project

14.4 Variante razvojnih modelov informacijskih sistemov

Poznamo različne razvojne modele:

- **zaporedni ali slapovni** (angl. *Waterfall*) (glej poglavje 14.4.1),
- **iterativni** (angl. *Iterative*) (glej poglavji 14.4.2 in 14.4.4),
- **prototipni** (angl. *Prototype*) (glej poglavji 14.4.3 in 14.4.4),
- **inkrementalni** (angl. *Incremental*) (glej poglavje 14.4.5),
- **kombinirani** (glej poglavje 14.4.7).

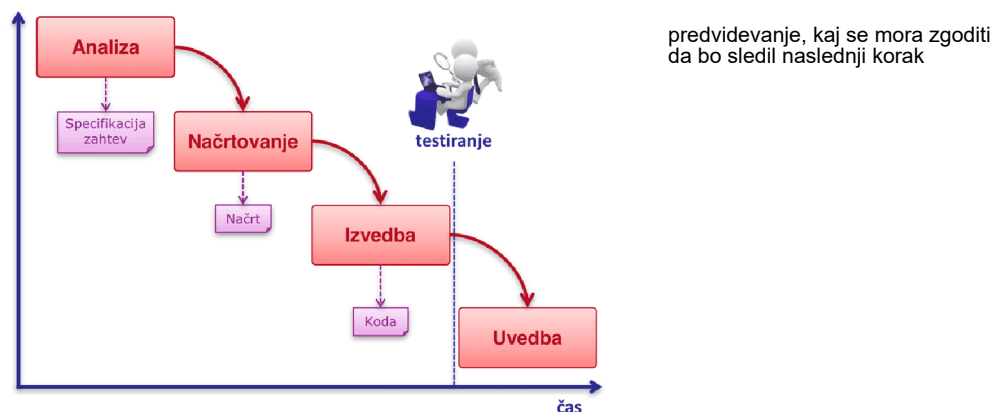
14.4.1 Zaporedni oz. slapovni razvojni model

Prednosti:

- Pomaga zmanjšati količino režijskega dela, ki ni v neposredni povezavi z izdelavo programske opreme (npr. vodenje projekta), saj je mogoče načrtovanje v celoti izvesti vnaprej.

Slabosti:

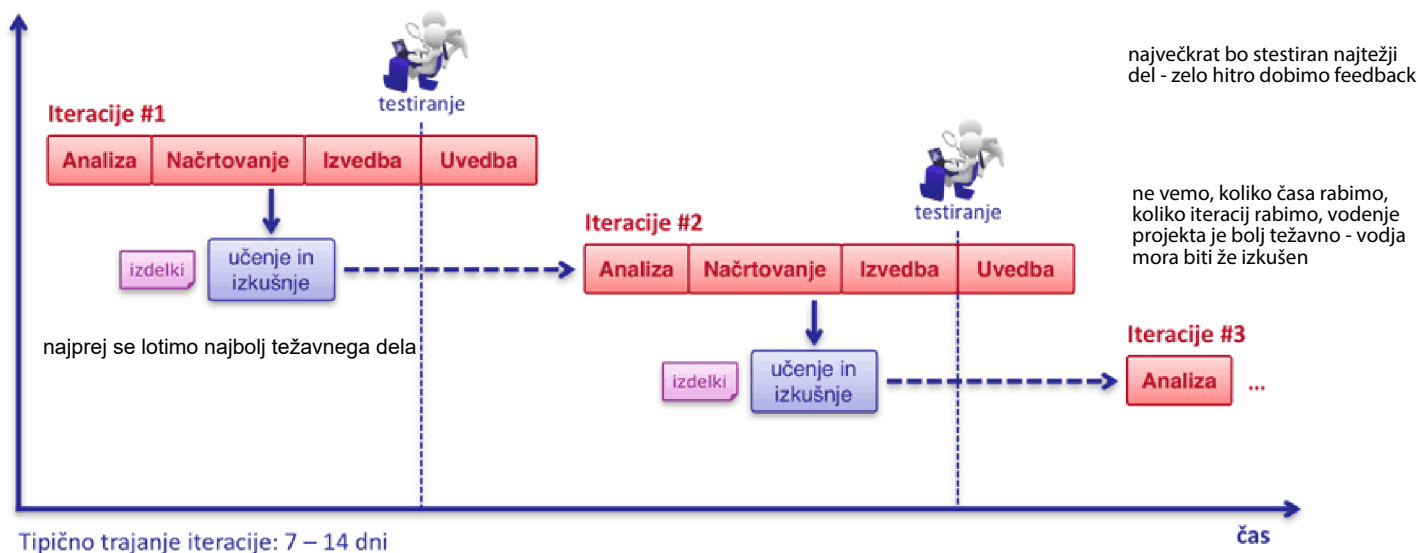
- Nefleksibilnost, saj naknadne spremembe zahtevajo veliko dodatnega napora.
- Nenaraven, saj je v praksi težko pričakovati, da se postopek v celoti zaključi pred začetkom naslednjega.
- Ne omogoča paralelnega izvajanja delov postopkov.



Slika 14.6: Zaporedni oz. slapovni razvojni model

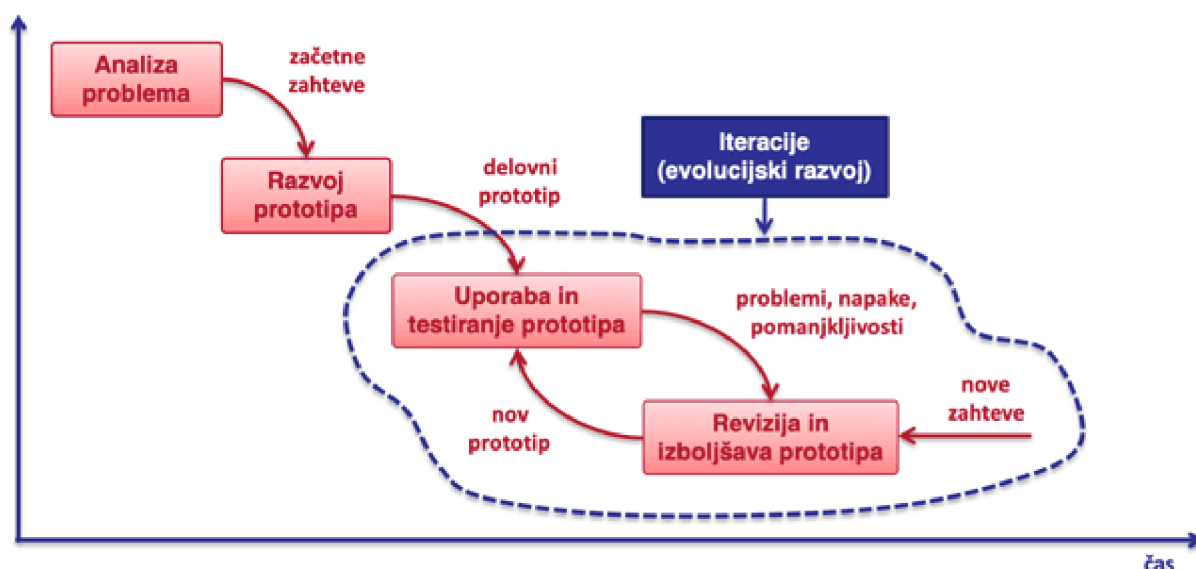
14.4.2 Iterativni razvojni model

Pri iterativnem razvojnem modelu pripravimo verzijo in jo potem iterativno izboljšujemo.



Slika 14.7: Iterativni razvojni model

14.4.3 Prototipni razvojni model



14.4.4 Iterativni/prototipni razvojni model

Prednosti iterativnega/prototipnega razvojnega modela v primerjavi z zaporednim oz. slapovnim razvojnim modelom:

- najbolj tvegani deli so razrešeni, še preden postane investicija velika,
- začetne iteracije omogočajo zgodnje povratne informacije s strani uporabnikov,
- preizkušanje in povezovanje v sistem sta nepretrgana,
- ciljni mejniki omogočajo kratkoročno osredotočenje,
- napredek merimo z ocenjevanjem izvedenega dela,
- možna je predaja izvedenega dela projekta, še preden je dokončan celoten projekt.

Slabosti:

- ne omogoča dobrega načrtovanja poteka projekta,
- ni mogoče točno predvideti, koliko iteracij bo potrebnih za razvoj dokončnega (dovolj dobrega) izdelka,
- vodenje projekta je zahtevno.

14.4.5 Inkrementalni razvojni model

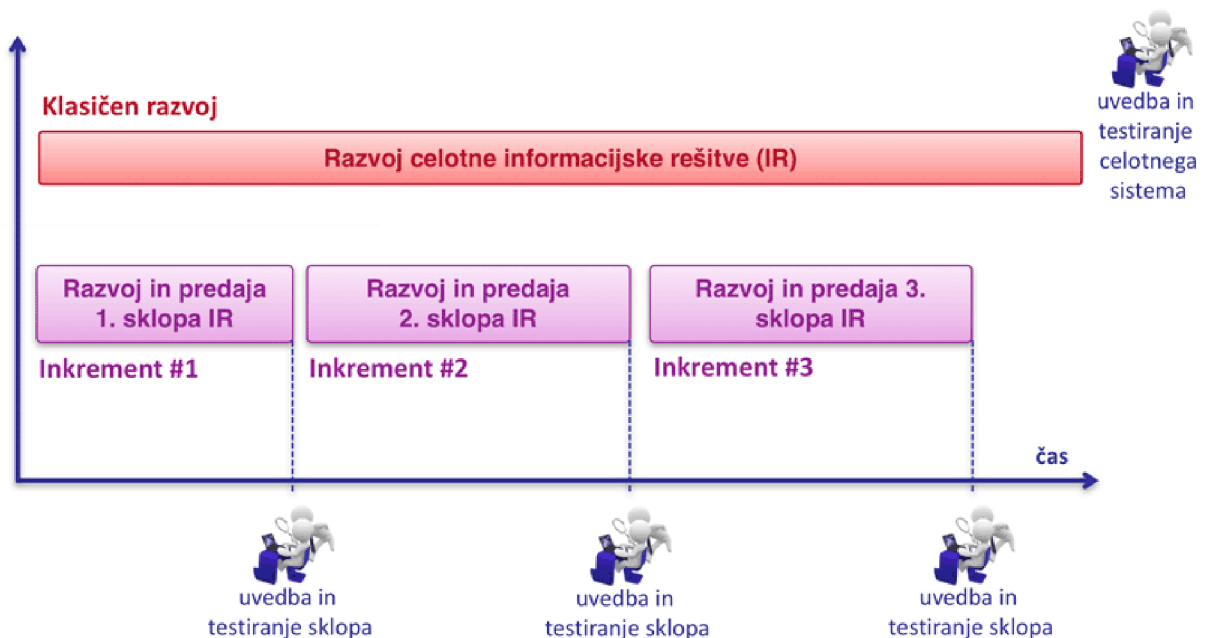
Pri inkrementalnem razvojnem modelu implementiramo del funkcionalnosti in ga predamo ter nato nadaljujemo z drugim delom funkcionalnosti.

Prednosti:

- uporabnik dobi del informacijske rešitve (IR) prej, saj se celota razvija po delih,
- rešitev, ki jo uporablja, se postopoma nadgrajuje, sam pa lahko sodeluje pri testiranju razvitih sklopov,
- naročnik lažje sledi napredovanju projekta.

Slabosti:

- ni možna uporaba na vseh vrstah projektov, saj nekaterih rešitev ni mogoče predati v uporabo po delih,
- IR moramo razdeliti na sklope in predvideti odvisnosti med njimi, pri neustreznem načrtovanju se lahko zgodi, da sklope neustrezno razporedimo.



Slika 14.9: Inkrementalni razvojni model

14.4.6 Iterativnost vs. inkrementalnost

Pri **iterativnem** procesu razvoja **verzijo iterativno izboljšujemo**.

Če nekaj naredimo zgolj enkrat, potem to ni iteracija.

Kot prikazuje slika 14.10, je delo večine slikarjev iterativne narave. Najprej si pripravijo začetno skico, nato dodajo barvni osnutek, potem pa sledi zaključevanje podrobnosti. Zaključijo takrat, ko so z rezultatom zadovoljni.



agile space - zelo viden vidik inkrementalnosti in iterativnosti

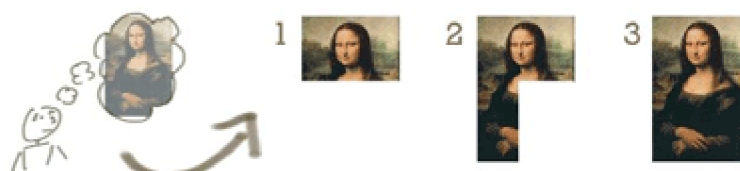
waterfall - nima ničesar od tega
staged delivery - inkrementalnost
spiral model - iterativnost

Slika 14.10: Iterativnost pri delu slikarja

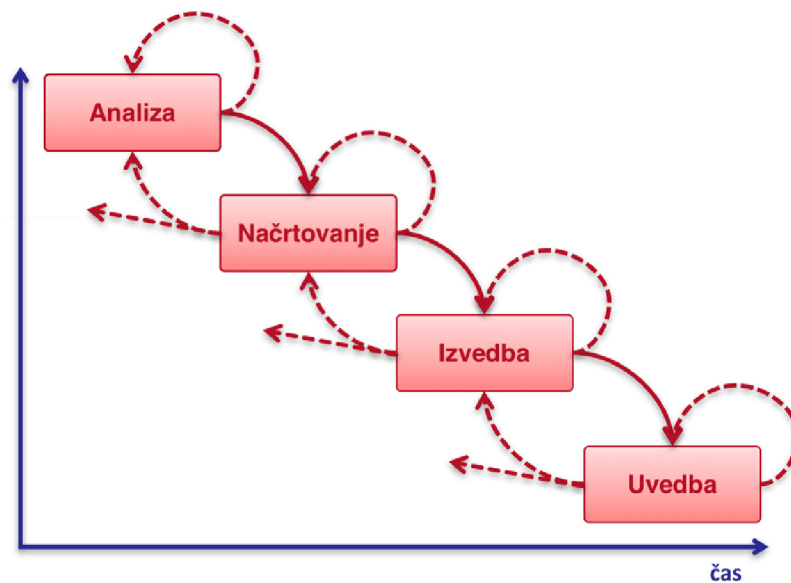
Pri **inkrementalnem** procesu razvoja **postopoma dodajamo** del posamezne funkcionalnosti programa, ki jo tudi **predamo stranki**.

Določen inkrement naredimo **zgolj enkrat**.

Analogija z delom slikarja, ki je v praksi sicer manj verjetna, je prikazana na sliki 14.11. Sliko bi izdelali po delih, npr. najprej v celoti glavo, potem dodamo še preostale dele. Podobno kot bi gradili opečnati zid, kjer po večjih inkrementih dobimo večji zid.



14.4.7 Kombinirani razvojni model praksa



Slika 14.12: Kombinirani razvojni model

14.5 Metodologije razvoja

14.5.1 Različne opredelitve v literaturi

Metodologija je priporočena zbirka filozofij, faz, postopkov, pravil, tehnik, orodij, dokumentacije, upravljanja in izobraževanja za razvijalce informacijskih sistemov.

— Maddison

Metodologija je množica dogovorov (konvencij), s katerimi se (projektna) skupina (organizacija) strinja.

— Cockburn, 2002

Metodologija je vse, kar redno izvajamo, da bi dosegli končni rezultat – delujoča programska oprema pri končnem uporabniku.

— Cockburn, 2002

Metodologija razvoja informacijskega sistema je priporočen način razvoja informacijskega sistema, ki temelji na filozofiji in množici principov.

— Avison, 2003

14.5.2 Metodologija v terminološkem slovarju

Metodologija je skupek metod, postopkov in standardov, ki sestavljajo zaključeno celoto pri izvajanju inženirskih pristopov k razvoju produkta.

Metoda je seznam postopkov in pravil za izvedbo določene naloge.

Metodologija razvoja informacijskega sistema je postopen način razvoja informacijskega sistema, ki vključuje uporabo različnih tehnik in orodij, in je celovit postopek v smislu korakov življenjskega cikla razvoja.

14.5.3 Naša opredelitev

Metodologija zajema vse, kar delamo, da bi dosegli željen rezultat:

- postopki, ki so neposredno povezani z razvojem,
- podporni postopki,
- način komunikacije med sodelujočimi,
- pravila odločanja.

Sociološke komponente metodologije:

- filozofija,
- načela,
- ideje,
- pogledi.

Razvojni oddelki **formalne metodologije** običajno prilagodijo po svoji meri, da ustrezajo njihovem načinu dela in razumevanju. **Uporabniki se z uporabo učijo** in nabirajo nove izkušnje.

Metodologija je **množica dogovorov**, s katerimi se projektna organizacija strinja.

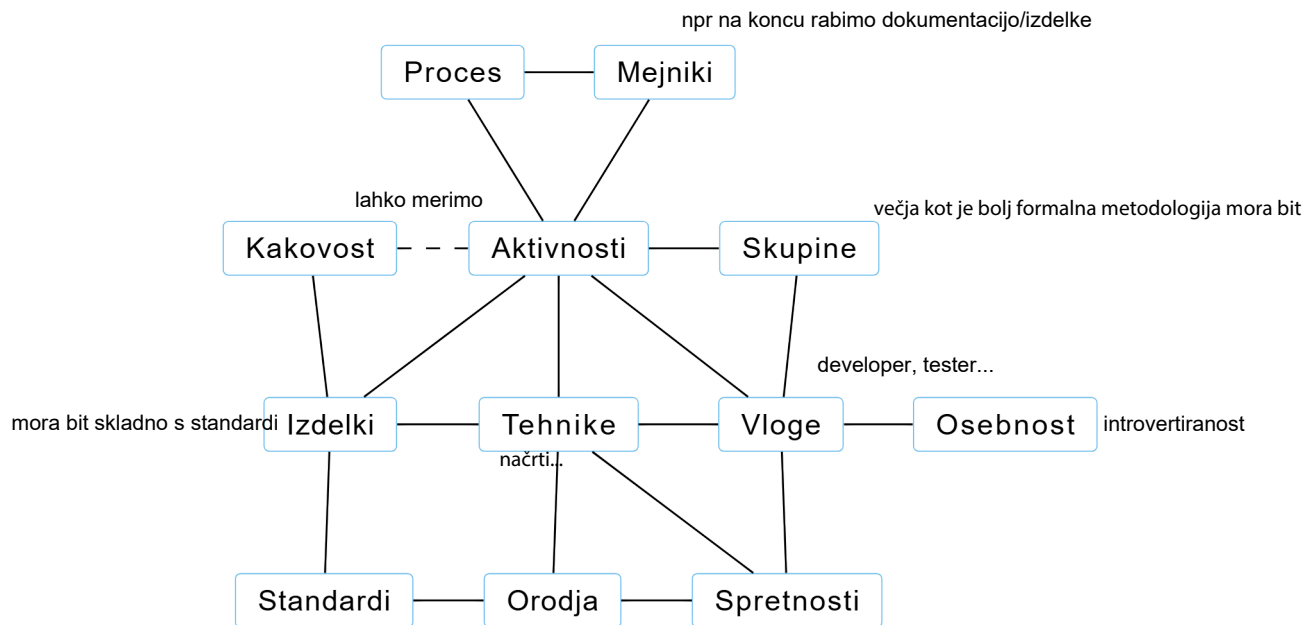
14.5.4 Elementi metodologije

Metodologijo sestavljajo **formalni**:

- postopki,
- pravila,
- napotki,
- smernice,
- standardi,
- ...

in **neformalni** elementi:

- nedokumentirani elementi,
- znanje, ki ga člani organizacije uporabljajo pri svojem delu,
- ...



Slika 14.13: Elementi metodologije agilnega razvoja PlantUML.

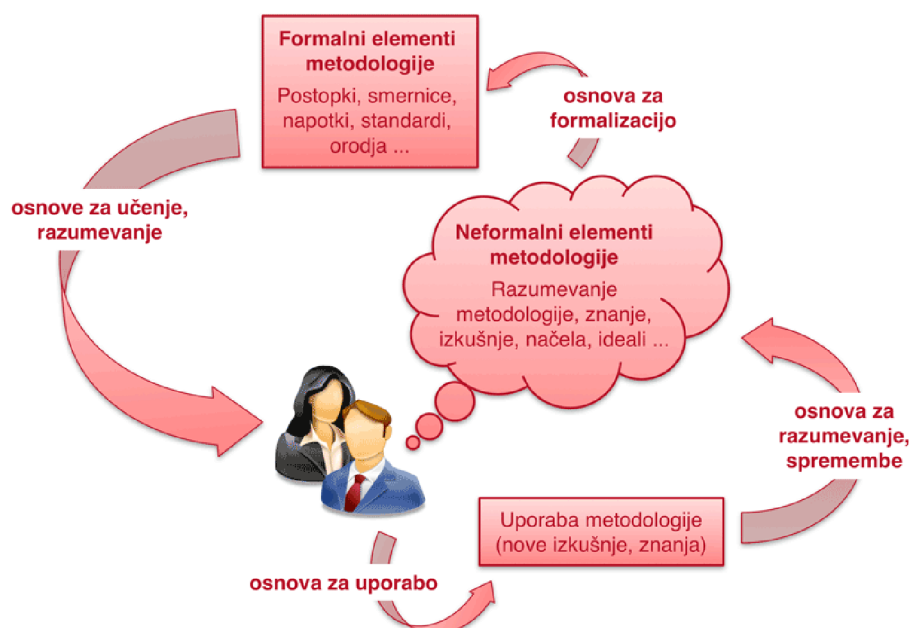
14.5.5 Upravljanje z znanjem

Iskanje tehnik in metod za zajem, formalizacijo in prenos znanja:

- **eksplicitno znanje** (angl. *explicit knowledge*) je moč izraziti v formalni obliki,
- **skrito/tacitno znanje** (angl. *tacit knowledge*) je subjektivne narave in prepleteno z izkušnjami, ideali, čustvi, intuicijo ipd. ter lahko s časom postane rutina.

Iz dokumentirane metodologije se uporabniki učijo in razvijejo svoje razumevanje načina dela, komunikacije in odločanja.

14.5.6 Formalna vs. neformalna metodologija



Slika 14.14: Od neformalne do formalne metodologije



en programira, drugi gleda čez ramo in preverja, narekuje... potem se zamenjata. ni statična - pari se menjajo

s tem zagotovimo, da več ljudi pozna celotno kodo

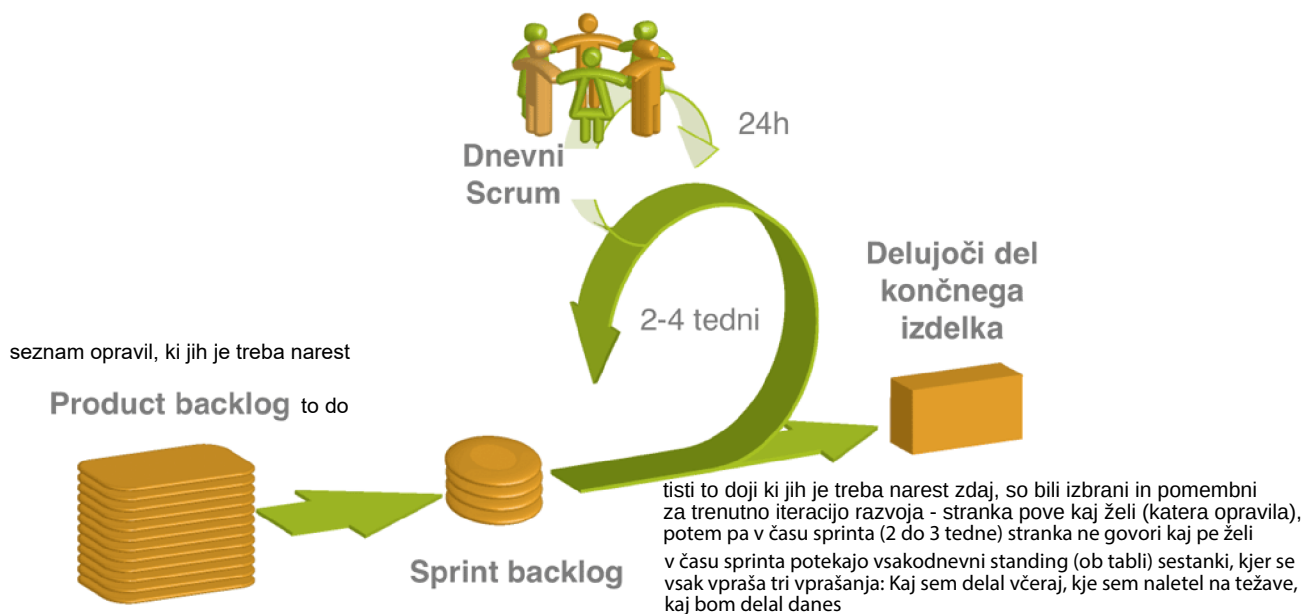
ta metodologija prepoveduje overtime - če se preveč dela metodologija dolgoročno ni produktivna



Slika 14.17: Pomembni gradniki metodologije XP

14.5.7.4 SCRUM

SCRUM je ena najbolj znanih metodologij za projektno vodenje po agilnem pristopu, kjer so osnovni gradniki prikazani na sliki 14.18. kjer je projektno vodenje težko, zato SCRUM daje nasvete, kako to delati



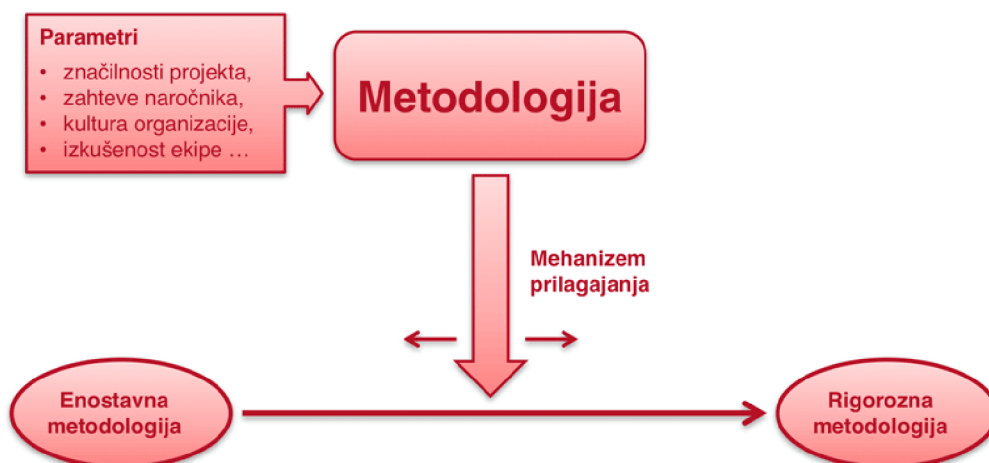
Slika 14.18: Pomembni gradniki metodologije SCRUM

Introduction to Scrum - 7 Minutes



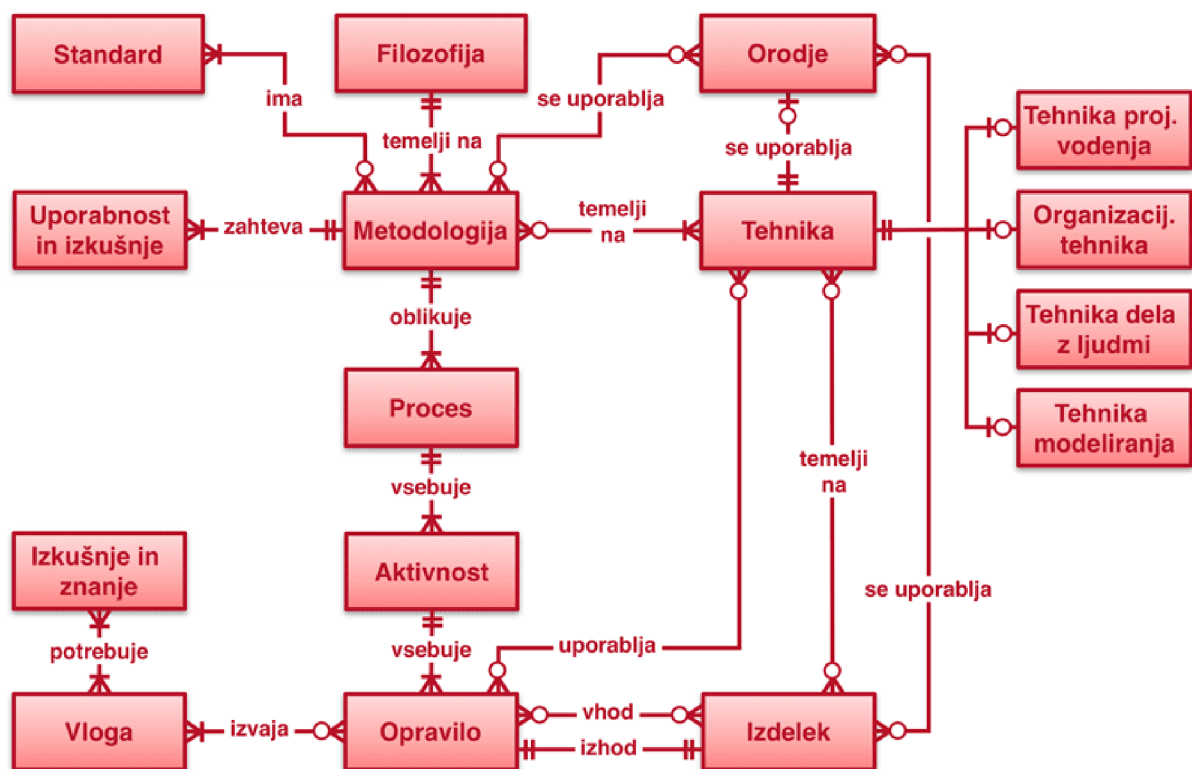
14.5.7.5 Osnovna načela agilnih metodologij

- Posamezniki in njihova komunikacija so pomembnejši kot sam proces in orodja.
- Delujoča programska oprema je pomembnejša kot popolna dokumentacija.
- Vključevanje uporabnika je pomembnejše kot pogajanje na osnovi pogodb.
- Upoštevanje sprememb je pomembnejše kot sledenje planu.

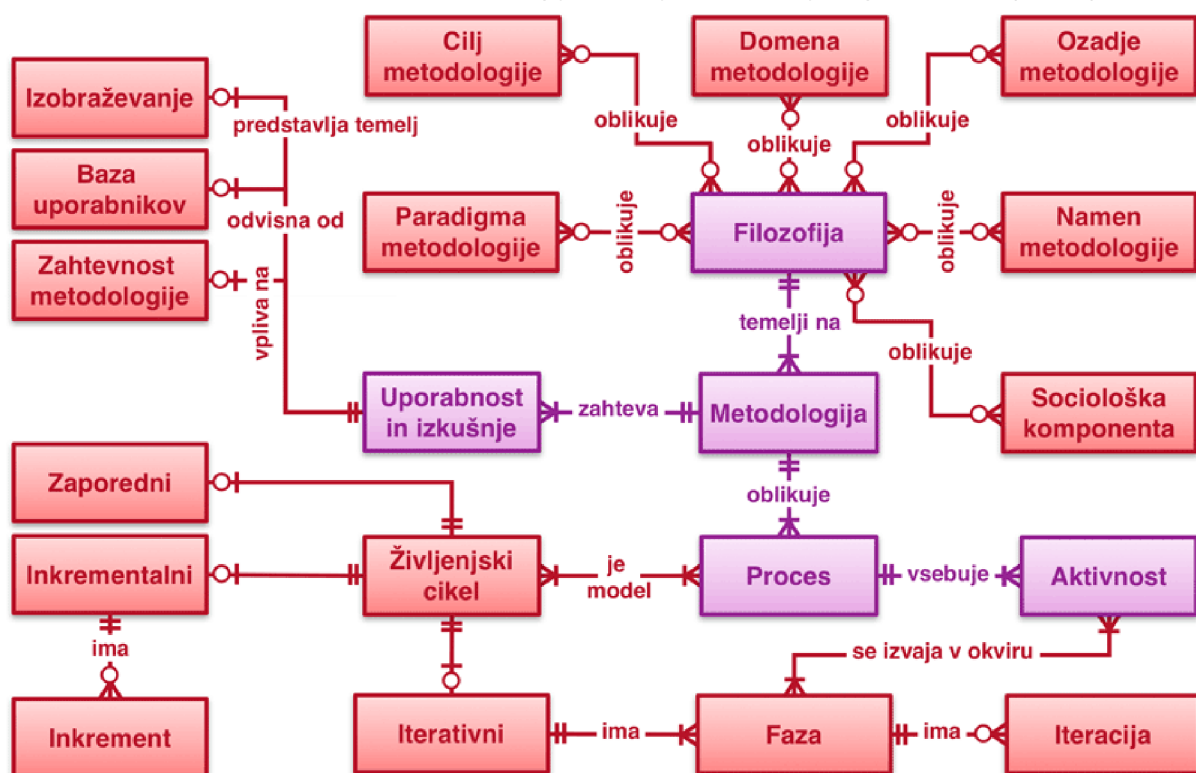


Slika 14.19: Koncept agilne metodologije

14.5.8 Metamodel metodologije razvoja informacijskega sistema



Slika 14.20: Metamodel metodologije razvoja informacijskega sistema (1. del)

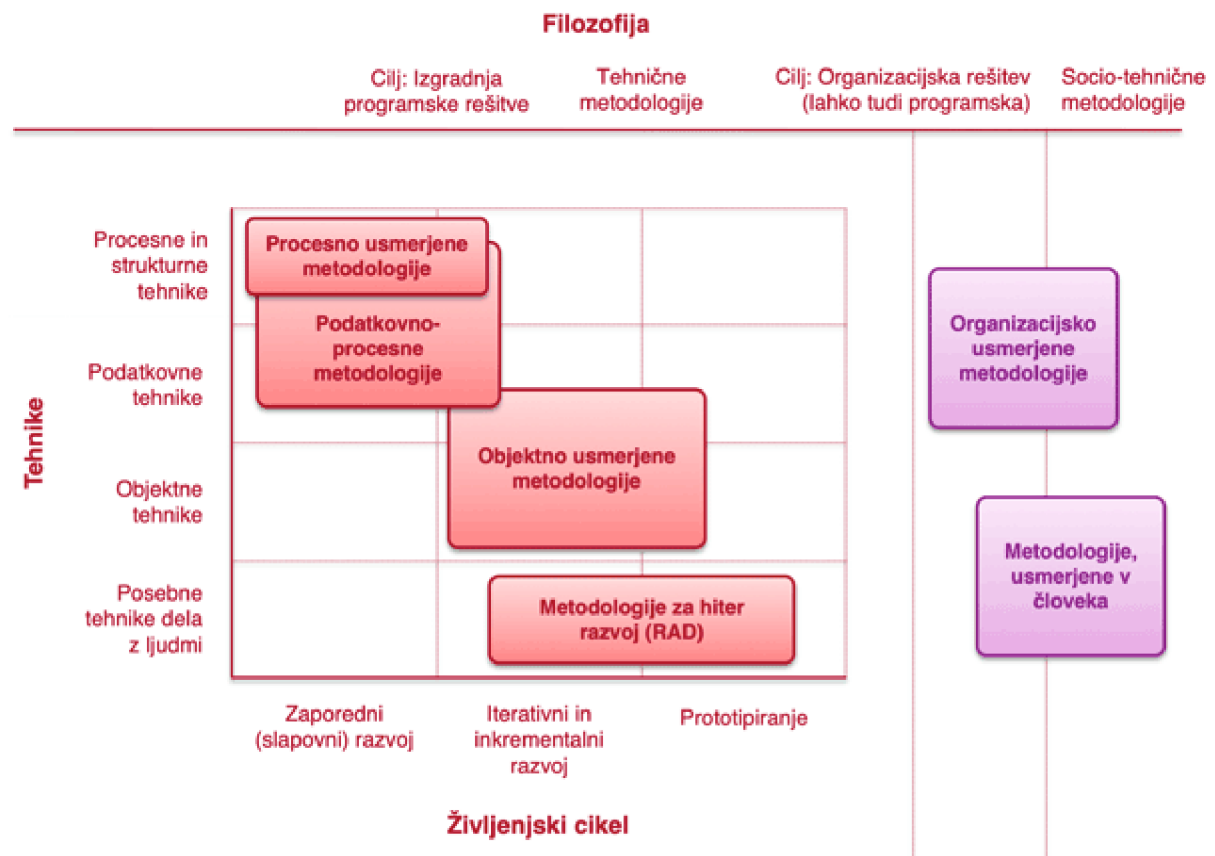


Slika 14.21: Metamodel metodologije razvoja informacijskega sistema (2. del)

14.5.9 Ogradja za primerjavo metodologij

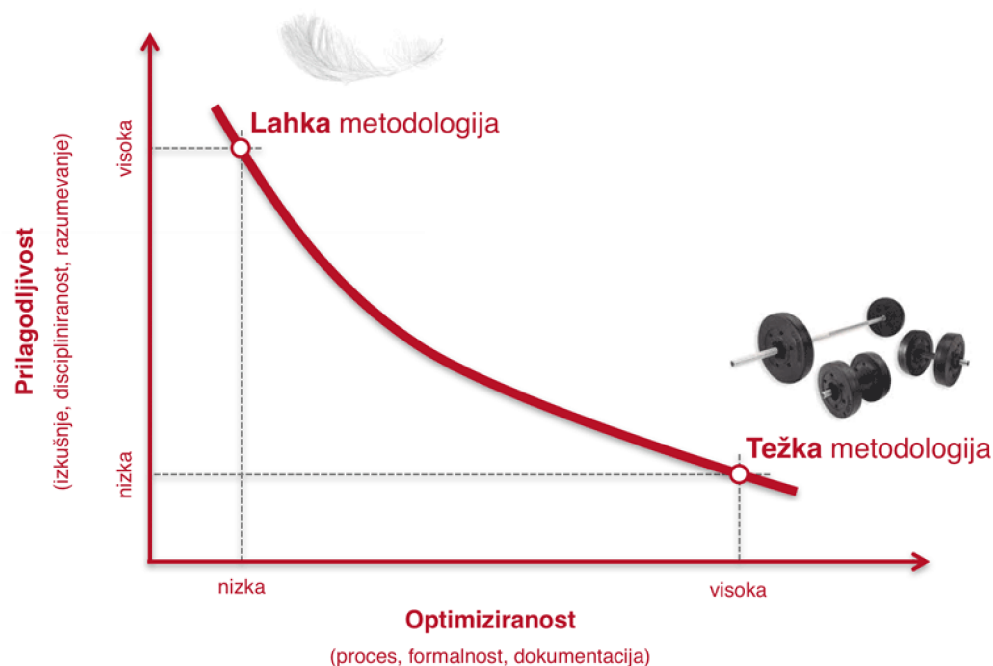
- **zgradba** kot osnova za ovrednotenje,
 - delitev glede na **tip** (glej poglavje 14.5.9.1),
 - delitev glede na **težo** (glej poglavje 14.5.9.2),
 - delitev glede na **utežitev** (glej poglavje 14.5.9.3),
 - evalvacija na **podlagi agilnih principov** (glej poglavje 14.5.9.4).

14.5.9.1 Delitev metodologij glede na tip



Slika 14.22: Delitev metodologij glede na tip

14.5.9.2 Delitev metodologij glede na težo



Slika 14.23: Delitev metodologij glede na težo

Težo metodologije opredeljujeta:

- **obseg**, ki predstavlja število različnih elementov, ki jih metodologija opisuje, in
- **gostota**, ki je zahtevan nivo podrobnosti oz. formalnosti.

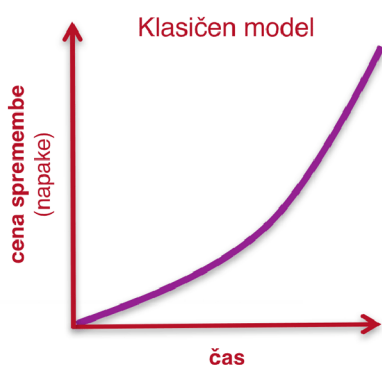
Tabela 14.1: Primerjava lahkih in težkih metodologij

Lahke metodologije	Težke metodologije
Glavni cilj je razvoj programske rešitve.	Cilj ni zgolj razvoj programskega sistema, ampak tudi prenova.
Imamo odgovorne, disciplinirane, izkušene in motivirane razvijalce, ki so seznanjeni s posebnimi tehnikami dela.	Imamo manj izkušene razvijalce, pri katerih točno opredeljena formalna pravila nadomeščajo izkušnje in znanje.
Naročnik, ki razume bistvo lahkih metodologij in je pripravljen sodelovati.	Naročnik zahteva visoko stopnjo formalizma.
Nepredvidljive in spreminjajoče se zahteve.	Imamo relativno dobro opredeljene in stabilne zahteve.
	Cilj je obsežnejši sistem z višjo stopnjo kritičnosti.

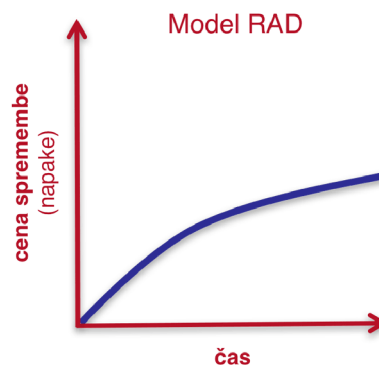
14.5.9.3 Delitev metodologij glede na utežitev

Tabela 14.2: Primerjava spredaj in zadaj uteženih metodologij

Spredaj utežene metodologije (glej sliko 14.24)	Zadaj utežene metodologije (glej sliko 14.25)
Poudarek dajejo postopkom analize in načrtovanja, ki se v veliki meri izvedejo vnaprej. začetne faze	Prednost dajejo postopkom izvedbe in testiranja, izdelani so morda le osnovni načrti. končne faze
Primerne so za razvoj: sistemov s stabilnimi zahtevami, kritičnih sistemov, obsežnih in kompleksnih sistemov z velikimi razvojnimi ekipami.	Primerne so za razvoj: manjših in bolj enostavnih sistemov z majhnimi in izkušenimi razvojnimi ekipami, sistemov s slabo določenimi zahtevami, ki se bodo še spreminjale, nekritičnih sistemov, sistemov, ki uporabljajo tehnologije, s katerimi nismo seznanjeni.
	Primeri so prototipiranje in lahke metodologije.



Slika 14.24: Klasičen model razvoja



Slika 14.25: Model RAD razvoja

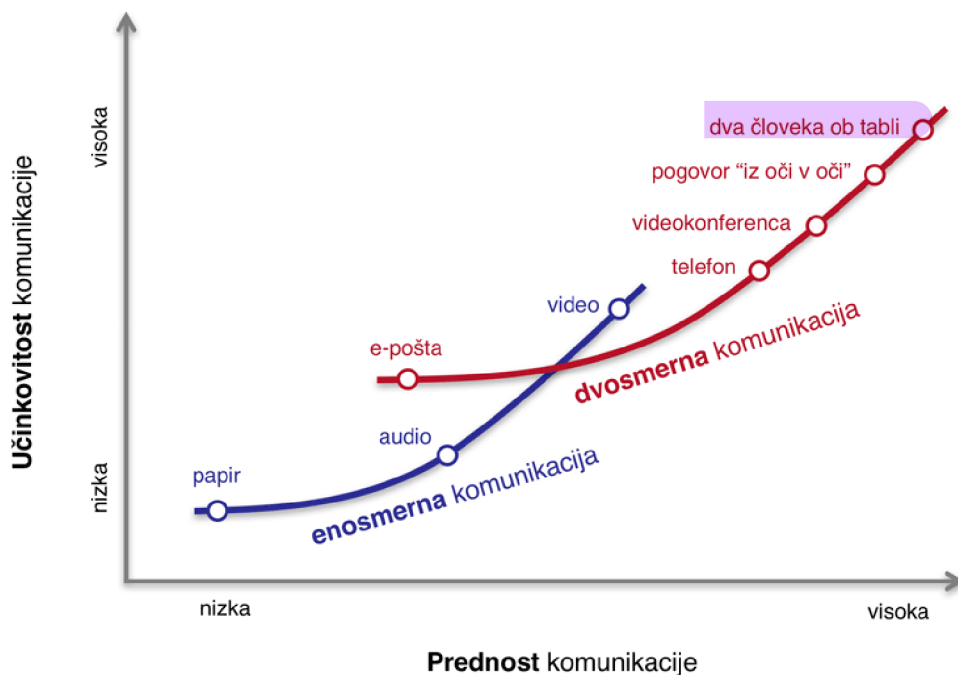
14.5.9.4 Ocenjevanje agilnih metodologij

- Neposredna komunikacija je najhitrejši in najcenejši kanal za izmenjavo informacij (glej sliko 14.26),
- nepotrebna dodatna teža metodologije je draga,
- večje razvojne skupine za svoje delovanje potrebujejo težje metodologije (glej sliko 14.28),
- višja stopnja formalnosti metodologije je primerna za projekte z višjo stopnjo kritičnosti (glej podrobnosti v 14.5.12),

- boljša komunikacija in **več povratnih informacij** zmanjšuje potrebo po vmesnih izdelkih (glej podrobnosti v 14.5.13),
- čim večje so **discipliniranost, izkušnje in znanje razvojne skupine**, tem **manjša je potreba po podrobno definiranem procesu**, formalnosti in dokumentaciji (glej podrobnosti v 14.5.15),
- odkloni od načrtovane rešitve nas vodijo k pravi rešitvi (glej podrobnosti v 14.5.16).

14.5.10 Učinkovitost različnih oblik komunikacije

Neposredna komunikacija je najhitrejši in najcenejši kanal za izmenjavo informacij.



Slika 14.26: Učinkovitosti in prednosti različnih oblik komunikacije

14.5.11 Vpliv velikosti na posebnosti razvojne skupine

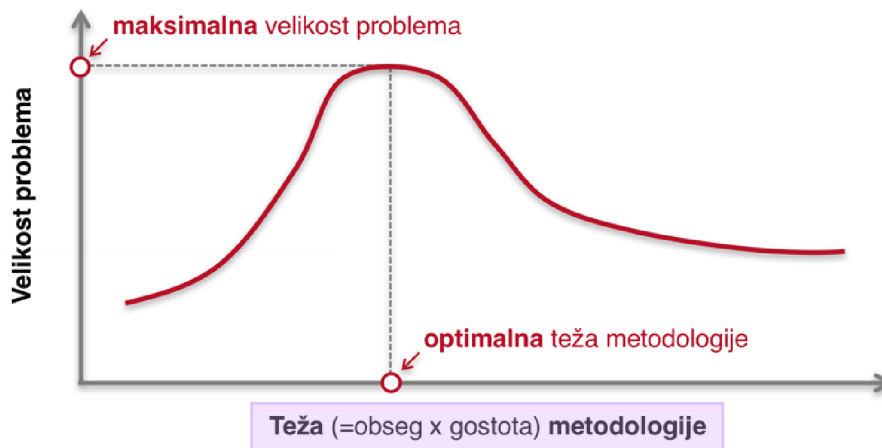
14.5.11.1 Majhna razvojna skupina



Slika 14.27: Posebnosti majhne razvojne skupine

Učinkovitost majhne razvojne skupine **z nepotrebnim povečanjem teže metodologije pada**.

14.5.11.2 Velika razvojna skupina



Slika 14.28: Posebnosti velike razvojne skupine

Večje razvojne skupine potrebujejo **težje metodologije**.

14.5.12 Kritični projekti

Za projekte z **višjo stopnjo kritičnosti** je primerna **višja stopnja formalnosti metodologije**.

Kritičnost sistema je **stopnja škode**, ki jo lahko **odpoved sistema** potencialno povzroči:

- izguba **udobja** (npr. nedelovanje spletne strani turistične agencije, zato ne moremo naročiti akcijske ponudbe za poletni oddih),
- izguba **denarja**,
 - nadomestljivega (npr. napaka v delovanju spletnega borznega ponudnika, kjer zamudimo z zahtevo za prodajo delnice),
 - nenadomestljivega,
- izguba **življenja** (npr. nedelovanje sistema za kontrolo letenja na letališču).

14.5.13 Komunikacija in povratne informacije

Boljša komunikacija in več povratnih informacij **zmanjšuje potrebo po vmesnih izdelkih**:

- hitra izdelava delujočega dela sistema, ki omogoča zgodnje preverjanje,
- zmanjšanje razvojne skupine in umestitev vseh njenih članov v skupen prostor,
- vmesni izdelki so namenjeni razvijalcem:
 - podroben projektni plan,
 - podroben načrt sistema,
 - podroben načrt testiranja.

14.5.14 Primer: Google: Inženirji vs. managerji

Že od ustanovitve podjetja  Google zaposleni v podjetju niso popolnoma zaupali v dodano vrednost managerjev.

We are a company built by engineers for engineers.

— Eric Flatt, software engineer

Leta 2002  Larry Page in  Sergey Brin poskušata eksperimentirati s **plosko organizacijo vodenja**, tako da se znebijo managerjev,

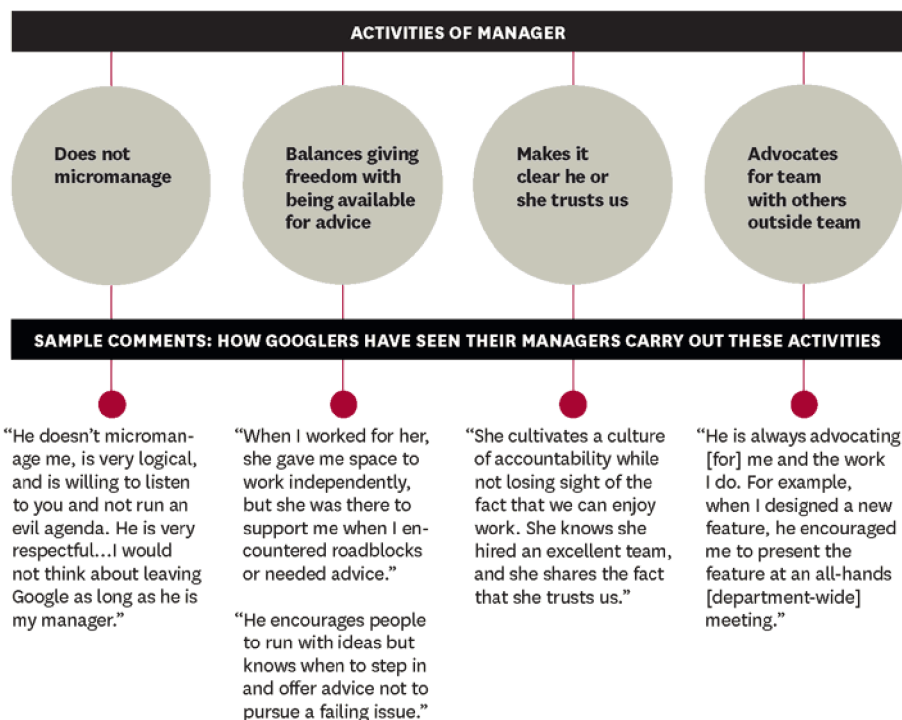
- trajalo je zgolj nekaj mesecev,
- npr. leta 2013 je bila porazdelitev 37.000 zaposlenih naslednja:
 - 5.000 managerjev (13,5 %) in
 - 1.000 direktorjev (2,7 %).

Kako prepričati visoko kvalificirani kader v dodano vrednost managerjev?

- Googlova rešitev: analitični pristop za testiranje merjenja njihovega vodstvenega kadra.
- V okviru **projekta Oxygen** jim ni uspelo potrditi, da so managerji nepotrebni.

Engineers **hate being micromanaged** on the technical side but love being closely managed on the career side.

— Garvin (2013):  How Google Sold Its Engineers on Management



Slika 14.29: Best practice: Assign stretch assignments to empower the team to tackle big problems (Garvin 2013)

How Google defines One Key Behavior: **Empowers the team and does not micromanage.**

14.5.15 Discipliniranost, izkušnje in znanje

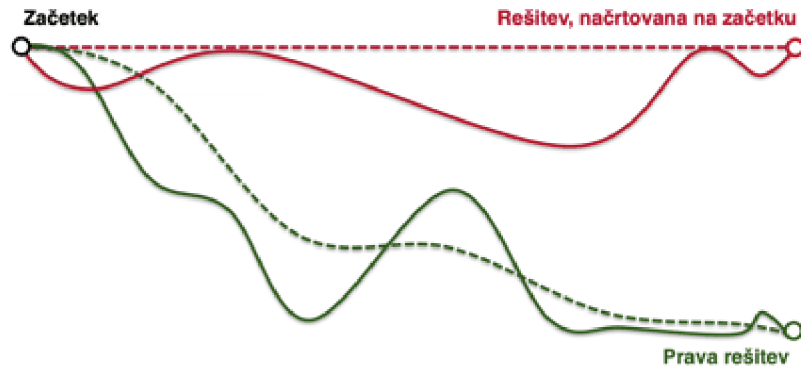
Čim večje so **discipliniranost, izkušnje in znanje razvojne skupine**, tem manjša je potreba po podrobno definiranem procesu, formalnosti in dokumentaciji:

- proces ≠ disciplina,
- formalnost ≠ izkušnje,

- dokumentacija $\neq$ znanje.

14.5.16 Odkloni od načrtovane rešitve

Zagovorniki agilnih pristopov trdijo, da nas odkloni od načrtovane rešitve vodijo k pravi rešitvi (glej sliko 14.30).



Slika 14.30: Odkloni od načrtovane rešitve

14.5.17 Evolucija razvoja programske opreme

Procesi:

- ad-hoc,
- slapovni,
- spiralni,
- inkrementalni,
- iterativni,
- RAD/JAD,
- Extreme Programming,
- Feature-Driven,
- Unified Process,
- ...

Tehnologije:

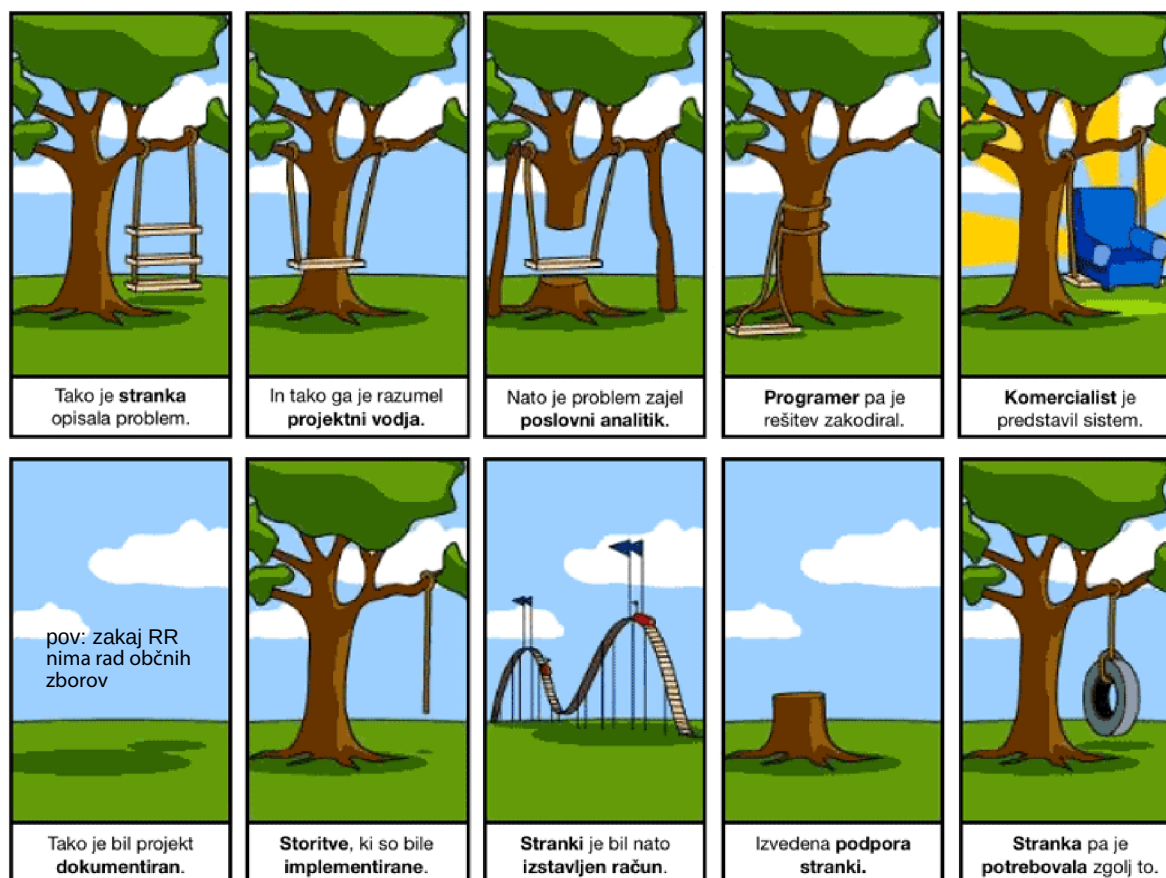
- jeziki,
 - zbirni,
 - proceduralni,
 - strukturirani,
 - objektni,
- 3G, 4G, CASE,
- orodja za podporo življenjskemu ciklu,
 - zajem zahtev,
 - načrtovanje,
 - izvedba,
 - testiranje,

- nadzor nad konfiguracijami/verzijami,
- modeliranje,
 - strukturirano,
 - DFD,
 - Coad, OMT,
 - UML,
 - ...

Še vedno se soočamo z istimi problemi kot pred 25 leti (glej sliko 14.31):

- slabo zastavljene zahteve,
- napeti roki,
- hitro spreminjajoča tehnologija,
- hitro spreminjajoče potrebe,
- na veliko zastavljeni projekti.

Izboljšave se uveljavljajo počasi.



Slika 14.31: Težave pri razvoju programske opreme

14.5.17.1 Kaj gre po navadi narobe?

Tabela 14.3: Kaj gre po navadi narobe?

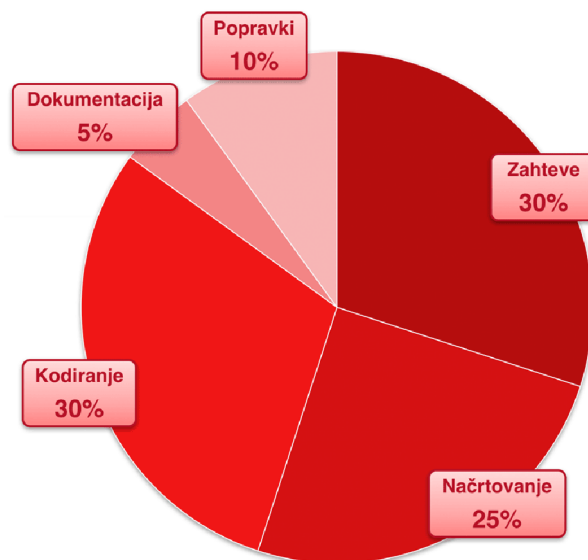
Delež	Težava
50%	Napačno postavljene zahteve.
30%	Napačno vodeno uvajanje sprememb.
25%	Slaba metodologija razvoja.
25%	Nerealni roki.
25%	Slabo vodenje projekta.
	Nezainteresirani uporabnik.

14.5.17.2 Porazdelitev napak v procesu razvoja

Večina napak se pojavi v fazah **zajema zahtev in načrtovanja** (več kot 50%, glej sliko 14.32). Pomembno je, da izpopolnimo postopek zajema zahtev z:

- vključitvijo zahtev v načrte programa,
- sprotnim testiranjem,
- objavljanjem sprotih oprijemljivih delovnih rezultatov,
- obvladovanjem neizogibnih sprememb.

Osredotočenost na stranke še vedno ostaja pomembna.



Slika 14.32: Porazdelitev napak v procesu razvoja

14.5.17.3 Inženirski pristop pri razvoju

Aplikacije morajo biti **zasnovane in načrtovane**, ne samo "izdelane".

Zgledujemo se lahko po inženirjih drugih panog (npr. gradbeništvo), kjer uporabljajo:

- procese, orodja, standarde,
- visoko izobražene, izurjene, potrjene kadre,

- koncept sistemske integracije.

Lastnosti **uspešnih projektov**:

- iterativni proces,
- spoštovanje zahtev,
- nenehna integracija, uporaba metrik in zagotavljanje kakovosti,
- delo s pogostimi, oprijemljivimi rezultati,
- skupinsko delo in podpora.